

Formal Verification of FiniteKernels.jl

Simon Frost

2026-09-07

Contents

FiniteKernelsProofs	1
What is formalised	1
Correspondence with the Julia API	1
FiniteKernelsProofs.Finite.Kernel	2
Conventions	2
FiniteKernelsProofs.Finite.Laws	9
FiniteKernelsProofs.Theory.Correspondence	12
FiniteKernelsProofs.Theory.FinStoch	14
Roadmap	17

FiniteKernelsProofs

Lean 4 / Mathlib formalisation accompanying FiniteKernels.jl (toolchain leanprover/lean4:v4.30.0, Mathlib tag v4.30.0; ADR 0005). This document is generated from the Lean sources by [mdgen](#): the prose is the module docstrings and the code blocks are the verbatim, machine-checked sources. Every library module is built by `lake build --wfail`; `Audit.lean` prints the axioms of the headline results (only `propext`, `Classical.choice`, `Quot.sound`). The Roadmap has no remaining proof holes.

What is formalised

FiniteKernels.jl implements finite stochastic kernels (`FiniteKernel`, tables in the outputs-first layout of ADR 0002), their sequential (`compose_kernel`) and parallel (`tensor_kernel`) composition and copy/discard/swap operations. `MarkovCategories.jl` exposes these as an instance of the GATlab theory `ThMarkovCategory` (SPEC §5–§6). The exact kernel equations are proved pointwise and are now bundled into a Mathlib `MarkovCategory` `FinStoch`, including the symmetric monoidal coherence laws. No external `Stoch` development is vendored and no correspondence with floating-point code is asserted.

- `Finite/Kernel.lean`: the finite model $\text{Kernel } X \ Y := X \rightarrow Y \rightarrow \mathbb{R}$, its operations, preservation of normalisation/nonnegativity, and the category, symmetric-monoidal and commutative-comonoid laws.
- `Finite/Laws.lean`: discard naturality iff normalised; copy naturality iff deterministic; “copy once is not two samples” for states.
- `Theory/Correspondence.lean`: the GATlab/Mathlib/finite-kernel dictionary, with examples checking the type of each Mathlib field.
- `Theory/FinStoch.lean`: concrete Mathlib category, monoidal, symmetric, comonoid and Markov instances; abstract copy naturality iff the kernel is deterministic. Empty objects are permitted without assuming nonexistent maps into them.
- `Roadmap.lean`: compatibility import and remaining representation questions; no unproved declarations.

Correspondence with the Julia API

Julia (FiniteKernels.jl)	Lean
FiniteKernel table table[y..., x...], kernel_matrix(k)[y, x] is_normalized(k) compose_kernel(k, l) (diagrammatic order) tensor_kernel(k, l) identity_kernel, copy_kernel, discard_kernel, swap_kernel deterministic(X, Y, f), point_mass(X, a), state(X, p) testset “category laws” testset “monoidal laws”	Kernel X Y, entry k x y = P(y x) Kernel.Normalised k Kernel.comp k l Kernel.tensor k l idK X, copy X, discard X, swap X Y ofFun f, pointMass a, State X comp_assoc, idK_comp, comp_idK comp_tensor, tensor_idK, tensor_assoc, tensor_unit_left, tensor_unit_right, swap_swap, tensor_swap, hexagon copy_assoc, copy_discard_left, copy_discard_right, copy_swap, copy_prod, discard_prod, copy_unit, discard_unit comp_discard_eq_discard_iff comp_copy_eq_iff_isDeterministic comp_copy_eq_tensor_iff_isPointMass
testsets “comonoid laws”, “coherence of copy and discard”	
“discard naturality iff normalised”	
“copy is natural for deterministic kernels only”	
“copy once is not two samples” (SPEC §6)	

SPEC §61’s Propositions 1–7 concern Bayesian networks and influence diagrams and are formalised in BayesianNetworks.jl/proofs and InfluenceDiagrams.jl/proofs; this project supplies the kernel-level laws those build on. The generic consequences of the abstract axioms (discard_natural as a theorem, state_discard, deterministic_comp, deterministic_copy) are stated once, for Mathlib’s classes, in BayesianNetworksProofs.Markov.Basic.

```
namespace FiniteKernelsProofs
```

```
/-- Smoke lemma so that the axiom audit always has a first line. -/  
theorem smoke : (1 : Nat) + 1 = 2 := rfl
```

```
end FiniteKernelsProofs
```

FiniteKernelsProofs.Finite.Kernel

```
import Mathlib.Data.Real.Basic  
import Mathlib.Algebra.BigOperators.Ring.Finset  
import Mathlib.Data.Fintype.BigOperators  
import Mathlib.Data.Fintype.Prod  
import Mathlib.Logic.Equiv.Basic  
import Mathlib.Algebra.Order.BigOperators.Group.Finset  
import Mathlib.Tactic.Ring
```

A concrete finite model of the structure implemented by FiniteKernels.jl (src/kernels.jl, src/composition.jl, src/copy_discard.jl): finite stochastic kernels between finite types, with sequential composition, tensor product, copy, discard and swap.

The laws here are stated directly about kernels, pointwise, with the monoidal structure maps (associator, unitors, tensor μ) written as *function-reindexing kernels* ofFun e for the obvious equivalences. Theory/FinStoch.lean packages them into Mathlib category instances.

Conventions

- Kernel X Y := X $\rightarrow$ Y $\rightarrow$ $\mathbb{R}$; k x y is the Julia entry P(y | x), i.e. table[y..., x...] in the outputs-first layout of ADR 0002 (kernel_matrix(k)[y, x]).
- Normalised k is is_normalized(k): every column k x sums to one.
- Entries live in $\mathbb{R}$ rather than $\mathbb{R}_{\geq 0}$; nonnegativity is the separate predicate Nonneg and is preserved by every construction, but none of the laws below need it.

- $\text{comp } k \ l$ is the diagrammatic composite $\text{compose}(k, l)$ (first k , then l), i.e. the matrix product $\text{kernel_matrix}(l) * \text{kernel_matrix}(k)$.
- $\text{tensor } k \ l$ is $\text{otimes}(k, l)$, the outer product of the tables.
- Deterministic kernels are ofFun f (Julia $\text{deterministic}(X, Y, f)$); the identity, copy, discard, swap and point masses are all of this form, which is what makes the structural laws one-line proofs.

```
namespace FiniteKernelsProofs.Finite

open Finset

/-- A finite kernel 'X → Y' is a real-valued table 'k x y = P(y | x)' ('FiniteKernel.table' in
the outputs-first layout, read as a function of the input first). -/
abbrev Kernel (X Y : Type) : Type := X → Y → ℝ

/-- A state 'I → X' ('state(X, probs)'): a kernel out of the monoidal unit 'Unit'. -/
abbrev State (X : Type) : Type := Kernel Unit X

namespace Kernel

variable {X Y Z W X1 X2 X3 Y1 Y2 Y3 Z1 Z2 : Type}
```

Predicates

```
/-- 'is_normalized(k)': each column sums to one. -/
def Normalised [Fintype Y] (k : Kernel X Y) : Prop := ∀ x, ∑ y, k x y = 1

/-- All entries are nonnegative. -/
def Nonneg (k : Kernel X Y) : Prop := ∀ x y, 0 ≤ k x y

/-- A stochastic kernel: nonnegative and normalised (a morphism of **FinStoch**). -/
def Stochastic [Fintype Y] (k : Kernel X Y) : Prop := Nonneg k ∧ Normalised k

/-- A deterministic kernel in the sense of the Julia tests: every column is a point mass. -/
def IsDeterministic [DecidableEq Y] (k : Kernel X Y) : Prop :=
  ∀ x, ∃ y, ∀ y', k x y' = if y' = y then 1 else 0

/-- A state is a point mass ('point_mass(X, a)') at some 'a'. -/
def IsPointMass [DecidableEq X] (p : State X) : Prop :=
  ∃ a, ∀ x, p () x = if x = a then 1 else 0
```

Constructions

```
/-- Sequential composition 'compose(k, l)' (diagrammatic order): '(k • l)(z | x) = ∑ y k(y | x) l(z |
  ↪ y)',
the matrix product 'kernel_matrix(l) * kernel_matrix(k)'. -/
def comp [Fintype Y] (k : Kernel X Y) (l : Kernel Y Z) : Kernel X Z :=
  fun x z => ∑ y, k x y * l y z

/-- Parallel composition 'otimes(k, l)': the outer product '(k ⊗ l)(y1, y2 | x1, x2) = k(y1 | x1)
  ↪ l(y2 | x2)'. -/
def tensor (k : Kernel X1 Y1) (l : Kernel X2 Y2) : Kernel (X1 × X2) (Y1 × Y2) :=
  fun x y => k x.1 y.1 * l x.2 y.2

/-- The deterministic kernel 'deterministic(X, Y, f)' induced by a function: 'P(y | x) = [y = f x]'.
  ↪ -/
def ofFun [DecidableEq Y] (f : X → Y) : Kernel X Y :=
  fun x y => if y = f x then 1 else 0

/-- The identity kernel 'id(X)' ('identity_kernel'). -/
def idK (X : Type) [DecidableEq X] : Kernel X X := ofFun id

/-- The copy map 'mcopy(X) = Δ(X) : X → X ⊗ X', 'Δ(x1, x2 | x) = [x1 = x][x2 = x]' ('copy_kernel').
  ↪ -/
def copy (X : Type) [DecidableEq X] : Kernel X (X × X) := ofFun fun x => (x, x)
```

```

/-- The discard map 'delete(X) =  $\phi(X) : X \rightarrow I$ ', the table of ones ('discard_kernel'). -/
def discard (X : Type) : Kernel X Unit := fun _ => 1

/-- The symmetry 'braid(X, Y) =  $\sigma(X, Y) : X \otimes Y \rightarrow Y \otimes X$ ' ('swap_kernel'). -/
def swap (X Y : Type) [DecidableEq X] [DecidableEq Y] : Kernel (X × Y) (Y × X) :=
  ofFun (Equiv.prodComm X Y)

/-- The point mass 'point_mass(X, a) =  $\text{dirac}(X, a) : I \rightarrow X$ '. -/
def pointMass [DecidableEq X] (a : X) : State X := ofFun fun _ => a

/-- Push a state through a kernel ('apply(k, p) = compose(p, k)'). -/
def push [Fintype X] (k : Kernel X Y) (p : State X) : State Y := comp p k

```

Structural reindexing kernels (the monoidal coherence data).

FiniteKernels.jl is strict (Julia tuples are flat), so these are all identities there; Mathlib's MonoidalCategory is not strict, and its fields refer to them by name.

```

/-- The associator ' $\alpha_{X Y Z} : (X \otimes Y) \otimes Z \rightarrow X \otimes (Y \otimes Z)$ ' as a kernel. -/
def assoc (X Y Z : Type) [DecidableEq X] [DecidableEq Y] [DecidableEq Z] :
  Kernel ((X × Y) × Z) (X × (Y × Z)) := ofFun (Equiv.prodAssoc X Y Z)

/-- The left unitor ' $\lambda_X : I \otimes X \rightarrow X$ ' as a kernel. -/
def leftUnitor (X : Type) [DecidableEq X] : Kernel (Unit × X) X := ofFun (Equiv.punitProd X)

/-- The inverse left unitor ' $(\lambda_X).inv : X \rightarrow I \otimes X$ ' as a kernel. -/
def leftUnitorInv (X : Type) [DecidableEq X] : Kernel X (Unit × X) := ofFun (Equiv.punitProd X).symm

/-- The right unitor ' $\rho_X : X \otimes I \rightarrow X$ ' as a kernel. -/
def rightUnitor (X : Type) [DecidableEq X] : Kernel (X × Unit) X := ofFun (Equiv.prodPUnit X)

/-- The inverse right unitor ' $(\rho_X).inv : X \rightarrow X \otimes I$ ' as a kernel. -/
def rightUnitorInv (X : Type) [DecidableEq X] : Kernel X (X × Unit) := ofFun (Equiv.prodPUnit X).symm

/-- Mathlib's ' $\text{tensor}_\mu X X Y Y : (X \otimes X) \otimes (Y \otimes Y) \rightarrow (X \otimes Y) \otimes (X \otimes Y)$ ', the middle swap
'id(X)  $\otimes \sigma(X, Y)$   $\otimes \text{id}(Y)$ ' of Catlab's coherence axiom for 'mcopy'. -/
def tensor $\mu$  (X Y : Type) [DecidableEq X] [DecidableEq Y] :
  Kernel ((X × X) × (Y × Y)) ((X × Y) × (X × Y)) :=
  ofFun fun p => ((p.1.1, p.2.1), (p.1.2, p.2.2))

/-- The unique map ' $I \otimes I \rightarrow I$ ' ('( $\lambda_I$ ).hom'). -/
def unitMul : Kernel (Unit × Unit) Unit := ofFun fun _ => ()

```

Pointwise evaluation lemmas

```

theorem ext_iff {k l : Kernel X Y} : k = l  $\leftrightarrow \forall x y, k x y = l x y$  := by
  simp [funext_iff]

@[simp] theorem comp_apply [Fintype Y] (k : Kernel X Y) (l : Kernel Y Z) (x : X) (z : Z) :
  comp k l x z =  $\sum y, k x y * l y z$  := rfl

@[simp] theorem tensor_apply (k : Kernel X1 Y1) (l : Kernel X2 Y2) (x : X1 × X2) (y : Y1 × Y2) :
  tensor k l x y = k x.1 y.1 * l x.2 y.2 := rfl

@[simp] theorem ofFun_apply [DecidableEq Y] (f : X → Y) (x : X) (y : Y) :
  ofFun f x y = if y = f x then 1 else 0 := rfl

@[simp] theorem discard_apply (x : X) (u : Unit) : discard X x u = 1 := rfl

theorem idK_apply [DecidableEq X] (x y : X) : idK X x y = if y = x then 1 else 0 := rfl

theorem copy_apply [DecidableEq X] (x x1 x2 : X) :
  copy X x (x1, x2) = if x1 = x  $\wedge$  x2 = x then 1 else 0 := by

```

```

simp [copy, Prod.ext_iff]

theorem swap_apply [DecidableEq X] [DecidableEq Y] (x : X) (y : Y) (y' : Y) (x' : X) :
  swap X Y (x, y) (y', x') = if y' = y ∧ x' = x then 1 else 0 := by
  simp [swap, Prod.ext_iff]

theorem pointMass_apply [DecidableEq X] (a x : X) : pointMass a () x = if x = a then 1 else 0 := rfl

/-- 'discard' is the deterministic kernel of the unique map to 'Unit'. -/
theorem discard_eq_ofFun : discard X = ofFun fun _ => () := by
  funext x u; simp

```

The deterministic-kernel calculus

Every structural morphism is ofFun $_$, and ofFun is a functor from functions to kernels that is compatible with tensor. Together with the two lemmas for composing an arbitrary kernel with ofFun on either side, these make the monoidal and comonoid laws mechanical.

```

/-- Precomposing with a deterministic kernel reindexes the input. -/
theorem comp_ofFun_left [Fintype Y] [DecidableEq Y] (f : X → Y) (l : Kernel Y Z) :
  comp (ofFun f) l = fun x z => l (f x) z := by
  funext x z
  simp [comp, ite_mul, Finset.sum_ite_eq']

/-- Postcomposing with the deterministic kernel of an equivalence reindexes the output. -/
theorem comp_ofEquiv_right [Fintype Y] [DecidableEq Z] (k : Kernel X Y) (e : Y ≃ Z) :
  comp k (ofFun e) = fun x z => k x (e.symm z) := by
  funext x z
  simp only [comp_apply, ofFun_apply, mul_ite, mul_one, mul_zero]
  rw [Finset.sum_eq_single (e.symm z)]
  · simp
  · intro b _ hb
    rw [if_neg]
    exact fun h => hb (e.symm_apply_eq.mpr h).symm
  · simp

/-- Postcomposing with a general deterministic kernel pushes mass forward. -/
theorem comp_ofFun_right [Fintype Y] [DecidableEq Z] (k : Kernel X Y) (g : Y → Z) :
  comp k (ofFun g) = fun x z => ∑ y, if z = g y then k x y else 0 := by
  funext x z
  simp [comp]

theorem ofFun_comp_ofFun [Fintype Y] [DecidableEq Y] [DecidableEq Z] (f : X → Y) (g : Y → Z) :
  comp (ofFun f) (ofFun g) = ofFun (g ∘ f) := by
  rw [comp_ofFun_left]; rfl

theorem tensor_ofFun [DecidableEq Y1] [DecidableEq Y2] (f : X1 → Y1) (g : X2 → Y2) :
  tensor (ofFun f) (ofFun g) = ofFun (Prod.map f g) := by
  funext x y
  rcases x with ⟨x1, x2⟩; rcases y with ⟨y1, y2⟩
  simp only [tensor_apply, ofFun_apply, Prod.map_apply, Prod.mk.injEq]
  split_ifs <;> simp_all

theorem ofFun_congr [DecidableEq Y] {f g : X → Y} (h : ∀ x, f x = g x) : ofFun f = ofFun g := by
  rw [funext h]

```

Normalisation and nonnegativity are preserved

```

theorem Normalised.comp [Fintype Y] [Fintype Z] {k : Kernel X Y} {l : Kernel Y Z}
  (hk : Normalised k) (hl : Normalised l) : Normalised (comp k l) := by
  intro x
  have hl' : ∀ y, ∑ z, l y z = 1 := hl
  simp only [comp_apply]
  rw [Finset.sum_comm]

```

```

simp [← Finset.mul_sum, hl']
exact hk x

theorem Normalised.tensor [Fintype Y1] [Fintype Y2] {k : Kernel X1 Y1} {l : Kernel X2 Y2}
  (hk : Normalised k) (hl : Normalised l) : Normalised (tensor k l) := by
  intro x
  simp only [tensor_apply]
  rw [Fintype.sum_prod_type, ← Finset.sum_mul_sum, hk, hl, one_mul]

theorem Normalised.ofFun [Fintype Y] [DecidableEq Y] (f : X → Y) : Normalised (ofFun f) := by
  intro x
  simp [Finset.sum_ite_eq']

theorem Normalised.idK [Fintype X] [DecidableEq X] : Normalised (idK X) := Normalised.ofFun _
theorem Normalised.copy [Fintype X] [DecidableEq X] : Normalised (copy X) := Normalised.ofFun _
theorem Normalised.discard : Normalised (discard X) := by intro x; simp
theorem Normalised.swap [Fintype X] [Fintype Y] [DecidableEq X] [DecidableEq Y] :
  Normalised (swap X Y) := Normalised.ofFun _
theorem Normalised.pointMass [Fintype X] [DecidableEq X] (a : X) : Normalised (pointMass a) :=
  Normalised.ofFun _

theorem Nonneg.comp [Fintype Y] {k : Kernel X Y} {l : Kernel Y Z} (hk : Nonneg k) (hl : Nonneg l) :
  Nonneg (comp k l) := fun x z =>
  Finset.sum_nonneg fun y _ => mul_nonneg (hk x y) (hl y z)

theorem Nonneg.tensor {k : Kernel X1 Y1} {l : Kernel X2 Y2} (hk : Nonneg k) (hl : Nonneg l) :
  Nonneg (tensor k l) := fun _ _ => mul_nonneg (hk _ _) (hl _ _)

theorem Nonneg.ofFun [DecidableEq Y] (f : X → Y) : Nonneg (ofFun f) := by
  intro _ _; simp only [ofFun_apply]; split_ifs <|> norm_num

theorem Nonneg.discard : Nonneg (discard X) := fun _ _ => zero_le_one

theorem Stochastic.comp [Fintype Y] [Fintype Z] {k : Kernel X Y} {l : Kernel Y Z}
  (hk : Stochastic k) (hl : Stochastic l) : Stochastic (comp k l) :=
  ⟨hk.1.comp hl.1, hk.2.comp hl.2⟩

theorem Stochastic.tensor [Fintype Y1] [Fintype Y2] {k : Kernel X1 Y1} {l : Kernel X2 Y2}
  (hk : Stochastic k) (hl : Stochastic l) : Stochastic (tensor k l) :=
  ⟨hk.1.tensor hl.1, hk.2.tensor hl.2⟩

theorem Stochastic.ofFun [Fintype Y] [DecidableEq Y] (f : X → Y) : Stochastic (ofFun f) :=
  ⟨Nonneg.ofFun f, Normalised.ofFun f⟩

```

Category laws (Julia testset “category laws”)

```

theorem comp_assoc [Fintype Y] [Fintype Z] (k : Kernel X Y) (l : Kernel Y Z) (m : Kernel Z W) :
  comp (comp k l) m = comp k (comp l m) := by
  funext x w
  simp only [comp_apply, Finset.sum_mul, Finset.mul_sum]
  rw [Finset.sum_comm]
  simp [mul_assoc]

theorem idK_comp [Fintype X] [DecidableEq X] (k : Kernel X Y) : comp (idK X) k = k := by
  rw [idK, comp_ofFun_left]; rfl

theorem comp_idK [Fintype Y] [DecidableEq Y] (k : Kernel X Y) : comp k (idK Y) = k := by
  rw [idK]; exact comp_ofEquiv_right k (Equiv.refl Y)

```

Monoidal laws (Julia testset “monoidal laws”)

```

/-- Interchange: `compose(otimes(k, l), otimes(k', l')) = otimes(compose(k, k'), compose(l, l'))`. -/
theorem comp_tensor [Fintype Y1] [Fintype Y2] (k : Kernel X1 Y1) (l : Kernel X2 Y2)

```

```

    (k' : Kernel Y1 Z1) (l' : Kernel Y2 Z2) :
      comp (tensor k l) (tensor k' l') = tensor (comp k k') (comp l l') := by
  funext x z
  simp only [comp_apply, tensor_apply, Fintype.sum_prod_type, Finset.sum_mul_sum]
  exact Finset.sum_congr rfl fun y1 _ => Finset.sum_congr rfl fun y2 _ => by ring

/-- 'otimes(id(X), id(Y)) = id(otimes(X, Y))' . -/
theorem tensor_idK [DecidableEq X] [DecidableEq Y] : tensor (idK X) (idK Y) = idK (X × Y) := by
  simp only [idK, tensor_ofFun]; rfl

/-- Associativity of the tensor, up to the associator:
'otimes(otimes(k, l), m) · α = α · otimes(k, otimes(l, m))' . -/
theorem tensor_assoc [Fintype X1] [Fintype X2] [Fintype X3] [Fintype Y1] [Fintype Y2] [Fintype Y3]
  [DecidableEq X1] [DecidableEq X2] [DecidableEq X3] [DecidableEq Y1] [DecidableEq Y2]
  [DecidableEq Y3] (k : Kernel X1 Y1) (l : Kernel X2 Y2) (m : Kernel X3 Y3) :
  comp (tensor (tensor k l) m) (assoc Y1 Y2 Y3) = comp (assoc X1 X2 X3) (tensor k (tensor l m)) :=
  by
  rw [assoc, assoc, comp_ofEquiv_right, comp_ofFun_left]
  funext x y
  rcases x with ⟨⟨x1, x2⟩, x3⟩; rcases y with ⟨y1, y2, y3⟩
  simp [mul_assoc]

/-- Right unit of the tensor, up to the right unitor: 'otimes(k, id(I)) · ρ = ρ · k' . -/
theorem tensor_unit_right [Fintype X] [Fintype Y] [DecidableEq X] [DecidableEq Y] (k : Kernel X Y) :
  comp (tensor k (idK Unit)) (rightUnitor Y) = comp (rightUnitor X) k := by
  rw [rightUnitor, rightUnitor, comp_ofEquiv_right, comp_ofFun_left]
  funext x y
  simp [idK]

/-- Left unit of the tensor, up to the left unitor: 'otimes(id(I), k) · λ = λ · k' . -/
theorem tensor_unit_left [Fintype X] [Fintype Y] [DecidableEq X] [DecidableEq Y] (k : Kernel X Y) :
  comp (tensor (idK Unit) k) (leftUnitor Y) = comp (leftUnitor X) k := by
  rw [leftUnitor, leftUnitor, comp_ofEquiv_right, comp_ofFun_left]
  funext x y
  simp [idK]

/-- The symmetry is an involution: 'braid(X, Y) · braid(Y, X) = id(X ⊗ Y)' . -/
theorem swap_swap [Fintype X] [Fintype Y] [DecidableEq X] [DecidableEq Y] :
  comp (swap X Y) (swap Y X) = idK (X × Y) := by
  rw [swap, swap, ofFun_comp_ofFun, idK]; rfl

/-- Naturality of the symmetry: 'otimes(k, l) · braid = braid · otimes(l, k)' . -/
theorem tensor_swap [Fintype X1] [Fintype X2] [Fintype Y1] [Fintype Y2] [DecidableEq X1]
  [DecidableEq X2] [DecidableEq Y1] [DecidableEq Y2] (k : Kernel X1 Y1) (l : Kernel X2 Y2) :
  comp (tensor k l) (swap Y1 Y2) = comp (swap X1 X2) (tensor l k) := by
  rw [swap, swap, comp_ofEquiv_right, comp_ofFun_left]
  funext x y
  simp [mul_comm]

/-- The hexagon identity ('SymmetricCategory'/'BraidedCategory.hexagon_forward'), which is the
associator-explicit form of the Julia test
'braid(X ⊗ Y, Z) = (id(X) ⊗ braid(Y, Z)) · (braid(X, Z) ⊗ id(Y))' . -/
theorem hexagon [Fintype X] [Fintype Y] [Fintype Z] [DecidableEq X] [DecidableEq Y]
  [DecidableEq Z] :
  comp (assoc X Y Z) (comp (swap X (Y × Z)) (assoc Y Z X)) =
  comp (tensor (swap X Y) (idK Z)) (comp (assoc Y X Z) (tensor (idK Y) (swap X Z))) := by
  simp only [assoc, swap, idK, tensor_ofFun, ofFun_comp_ofFun]
  rfl

/-- 'braid(X, I) = id(X)' up to the unitors. -/
theorem swap_unit_right [Fintype X] [DecidableEq X] :
  comp (swap X Unit) (leftUnitor X) = rightUnitor X := by
  simp only [swap, leftUnitor, rightUnitor, ofFun_comp_ofFun]
  rfl

```

Comonoid laws (Julia testsets “comonoid laws” and “coherence of copy and discard”).

Each is named after the Mathlib field it instantiates; see Theory/Correspondence.lean.

```

/-- Coassociativity, 'ComonObj.comul_assoc': ' $\Delta \triangleright (X \triangleleft \Delta) = \Delta \triangleright (\Delta \triangleright X) \triangleright \alpha$ '. -/
theorem copy_assoc [Fintype X] [DecidableEq X] :
  comp (copy X) (tensor (idK X) (copy X)) =
    comp (comp (copy X) (tensor (copy X) (idK X))) (assoc X X X) := by
  simp only [copy, idK, assoc, tensor_ofFun, ofFun_comp_ofFun]
  rfl

/-- Left counit, 'ComonObj.counit_comul': ' $\Delta \triangleright (\varepsilon \triangleright X) = (\lambda_ X).inv$ '. -/
theorem copy_discard_left [Fintype X] [DecidableEq X] :
  comp (copy X) (tensor (discard X) (idK X)) = leftUnitorInv X := by
  simp only [copy, idK, discard_eq_ofFun, leftUnitorInv, tensor_ofFun, ofFun_comp_ofFun]
  rfl

/-- Right counit, 'ComonObj.comul_counit': ' $\Delta \triangleright (X \triangleleft \varepsilon) = (\rho_ X).inv$ '. -/
theorem copy_discard_right [Fintype X] [DecidableEq X] :
  comp (copy X) (tensor (idK X) (discard X)) = rightUnitorInv X := by
  simp only [copy, idK, discard_eq_ofFun, rightUnitorInv, tensor_ofFun, ofFun_comp_ofFun]
  rfl

/-- The strict form of the counit laws checked in Julia: ' $\Delta(X) \cdot (\phi(X) \otimes id(X)) = id(X)$ '. -/
theorem copy_discard_left_strict [Fintype X] [DecidableEq X] :
  comp (comp (copy X) (tensor (discard X) (idK X))) (leftUnitor X) = idK X := by
  rw [copy_discard_left, leftUnitorInv, leftUnitor, ofFun_comp_ofFun, idK]
  exact ofFun_congr fun x => by simp

/-- ' $\Delta(X) \cdot (id(X) \otimes \phi(X)) = id(X)$ '. -/
theorem copy_discard_right_strict [Fintype X] [DecidableEq X] :
  comp (comp (copy X) (tensor (idK X) (discard X))) (rightUnitor X) = idK X := by
  rw [copy_discard_right, rightUnitorInv, rightUnitor, ofFun_comp_ofFun, idK]
  exact ofFun_congr fun x => by simp

/-- Cocommutativity, 'IsCommComonObj.comul_comm': ' $\Delta \triangleright \beta = \Delta$ '. -/
theorem copy_swap [Fintype X] [DecidableEq X] : comp (copy X) (swap X X) = copy X := by
  simp only [copy, swap, ofFun_comp_ofFun]
  rfl

/-- Coherence of copy with the tensor, 'CopyDiscardCategory.copy_tensor':
' $\Delta[X \otimes Y] = (\Delta[X] \otimes \Delta[Y]) \triangleright \text{tensor}\mu$ ', Catlab's ' $\Delta(A \otimes B) = (\Delta(A) \otimes \Delta(B)) \cdot (id(A) \otimes \sigma(A,B) \otimes id(B))$ '.
-/-
theorem copy_prod [Fintype X] [Fintype Y] [DecidableEq X] [DecidableEq Y] :
  copy (X × Y) = comp (tensor (copy X) (copy Y)) (tensorμ X Y) := by
  simp only [copy, tensorμ, tensor_ofFun, ofFun_comp_ofFun]
  rfl

/-- Coherence of discard with the tensor, 'CopyDiscardCategory.discard_tensor':
' $\varepsilon[X \otimes Y] = (\varepsilon[X] \otimes \varepsilon[Y]) \triangleright (\lambda_ I).hom$ ', Catlab's ' $\phi(A \otimes B) = \phi(A) \otimes \phi(B)$ '. -/
theorem discard_prod [Fintype X] [Fintype Y] :
  discard (X × Y) = comp (tensor (discard X) (discard Y)) unitMul := by
  funext x u
  simp [unitMul]

/-- 'CopyDiscardCategory.copy_unit': ' $\Delta[I] = (\lambda_ I).inv$ ', Catlab's ' $\Delta(munit()) = id(munit())$ '. -/
theorem copy_unit : copy Unit = leftUnitorInv Unit := rfl

/-- 'CopyDiscardCategory.discard_unit': ' $\varepsilon[I] = 1 I$ ', Catlab's ' $\phi(munit()) = id(munit())$ '. -/
theorem discard_unit : discard Unit = idK Unit := by
  funext x u; simp [idK]

/-- Every deterministic kernel is a comonoid homomorphism: it commutes with copy
('Deterministic.copy_natural', the equation the Julia test checks for 'deterministic(X, Y, f)'). -/
theorem ofFun_copy_natural [Fintype X] [Fintype Y] [DecidableEq X] [DecidableEq Y] (f : X → Y) :
  comp (ofFun f) (copy Y) = comp (copy X) (tensor (ofFun f) (ofFun f)) := by
  simp only [copy, tensor_ofFun, ofFun_comp_ofFun]

```

```

rfl

/-- ... and with discard ('Deterministic.discard_natural'). -/
theorem ofFun_discard_natural [Fintype X] [Fintype Y] [DecidableEq Y] (f : X → Y) :
  comp (ofFun f) (discard Y) = discard X := by
  rw [comp_ofFun_left]; rfl

end Kernel

end FiniteKernelsProofs.Finite

```

FiniteKernelsProofs.Finite.Laws

```

import FiniteKernelsProofs.Finite.Kernel
import Mathlib.Tactic.NormNum
import Mathlib.Tactic.Linarith

```

The two facts that `FiniteKernels.jl` pins numerically in `test/test_laws.jl` and that separate a Markov category from a cartesian one, proved exactly for the finite model of `Finite/Kernel.lean`:

- **Discard naturality iff normalised** (`comp_discard_eq_discard_iff`): the Markov axiom $f \cdot \diamond(B) == \diamond(A)$ holds for a table precisely when it is normalised. Julia testset “discard naturality iff normalised”, including the $[0.5 \ 0.5; 1.0 \ 0.5]$ counterexample (`comp_discard_apply` gives the column sums $[1.5, 1.0]$).
- **Copy naturality iff deterministic** (`comp_copy_eq_iff_isDeterministic`): for a normalised kernel, $f \cdot \Delta(B) == \Delta(A) \cdot (f \otimes f)$ holds iff every column is a point mass. Julia: “copy is natural for deterministic kernels only”.
- **Copy versus tensor** (`comp_copy_eq_tensor_iff_isPointMass`): the special case of a state $p : I \rightarrow X$; $\text{compose}(p, \text{mcopy}(X)) = \text{otimes}(p, p)$ (modulo $I \otimes I \cong I$, which is `copy Unit`) iff p is a point mass. Julia testset “copy once is not two samples (SPEC section 6)”.

The one piece of real arithmetic is `exists_pointMass_of_mul`: if $q \ x \ * \ q \ x' = [x = x'] \ q \ x$ and $\sum q = 1$, then q is an indicator. From $q \ x \ * \ q \ x = q \ x$ one gets $q \ x \in \{0, 1\}$; the sum forces at least one 1; and $q \ x \ * \ q \ a = 0$ for $x \neq a$ forces the rest to be 0.

```

namespace FiniteKernelsProofs.Finite.Kernel

open Finset

variable {X Y : Type}

```

Discard naturality iff normalised

```

/-- 'compose(k, delete(Y))' is the table of column sums. -/
theorem comp_discard_apply [Fintype Y] (k : Kernel X Y) (x : X) (u : Unit) :
  comp k (discard Y) x u =  $\sum y, k \ x \ y$  := by
  simp

/-- The Markov axiom 'f ·  $\diamond(B)$  ==  $\diamond(A)$ ' holds for a table iff the table is normalised. -/
theorem comp_discard_eq_discard_iff [Fintype Y] (k : Kernel X Y) :
  comp k (discard Y) = discard X  $\leftrightarrow$  Normalised k := by
  constructor
  · intro h x
    have := congrFun (congrFun h x) ()
    simpa using this
  · intro h
    funext x u
    simp [h x]

/-- Discard is natural for normalised kernels ('MarkovCategory.discard_natural'). -/
theorem Normalised.comp_discard [Fintype Y] {k : Kernel X Y} (hk : Normalised k) :
  Kernel.comp k (Kernel.discard Y) = Kernel.discard X :=

```

```

(comp_discard_eq_discard_iff k).mpr hk

/-- A normalised state has total mass one ('state_discard' in 'BayesianNetworksProofs'). -/
theorem Normalised.state_discard [Fintype X] {p : State X} (hp : Normalised p) :
  Kernel.comp p (Kernel.discard X) = Kernel.idK Unit := by
  rw [hp.comp_discard, discard_unit]

```

The indicator lemma

```

/-- If 'q x * q x' = [x = x'] q x' and 'Σ q = 1' then 'q' is the indicator of a single point. -/
theorem exists_pointMass_of_mul [Fintype X] [DecidableEq X] (q : X → ℝ) (hsum : Σ x, q x = 1)
  (h : ∀ x x', q x * q x' = if x = x' then q x else 0) :
  ∃ a, ∀ x, q x = if x = a then 1 else 0 := by
  have h01 : ∀ x, q x = 0 ∨ q x = 1 := by
    intro x
    have hx := h x x
    rw [if_pos rfl] at hx
    have : q x * (q x - 1) = 0 := by rw [mul_sub, mul_one, hx, sub_self]
    rcases mul_eq_zero.mp this with h0 | h1
    · exact Or.inl h0
    · exact Or.inr (sub_eq_zero.mp h1)
  have hex : ∃ a, q a = 1 := by
    by_contra hne
    push Not at hne
    have hzero : ∀ x, q x = 0 := fun x => (h01 x).resolve_right (hne x)
    have : Σ x, q x = 0 := Finset.sum_eq_zero fun x _ => hzero x
    rw [this] at hsum
    exact zero_ne_one hsum
  obtain ⟨a, ha⟩ := hex
  refine ⟨a, fun x => ?_⟩
  by_cases hxa : x = a
  · subst hxa; simp [ha]
  · have hx := h x a
    rw [if_neg hxa, ha, mul_one] at hx
    simp [hxa, hx]

/-- Conversely, an indicator satisfies the product identity. -/
theorem mul_ite_eq_of_pointMass [DecidableEq X] (a x x' : X) :
  (if x = a then (1 : ℝ) else 0) * (if x' = a then 1 else 0) =
  if x = x' then (if x = a then 1 else 0) else 0 := by
  by_cases hx : x = a
  · subst hx
    by_cases hx' : x' = x
    · subst hx'; simp
    · simp [hx', Ne.symm hx']
  · by_cases hx' : x' = a
    · subst hx'; simp [hx]
    · simp [hx, hx']

```

Copy naturality iff deterministic

```

/-- 'compose(k, mcopy(Y))' puts the column of 'k' on the diagonal (draw once, then copy). -/
theorem comp_copy_apply [Fintype Y] [DecidableEq Y] (k : Kernel X Y) (x : X) (y1 y2 : Y) :
  comp k (copy Y) x (y1, y2) = if y1 = y2 then k x y1 else 0 := by
  simp only [comp_apply, copy, ofFun_apply, Prod.mk.injEq, mul_ite, mul_one, mul_zero]
  by_cases h : y1 = y2
  · subst h
    simp
  · rw [if_neg h]
    apply Finset.sum_eq_zero
    intro y _
    rw [if_neg]
    rintro ⟨rfl, rfl⟩

```

```

exact h rfl

/-- 'compose(mcopy(X), otimes(k, k))' is the outer product of the column with itself
(draw twice, independently). -/
theorem comp_copy_tensor_apply [Fintype X] [DecidableEq X] (k : Kernel X Y) (x : X) (y1 y2 : Y) :
  comp (copy X) (tensor k k) x (y1, y2) = k x y1 * k x y2 := by
  simp [copy]

/-- For a normalised kernel, 'f · Δ(B) == Δ(A) · (f ⊗ f)' holds iff 'f' is deterministic. -/
theorem comp_copy_eq_iff_isDeterministic [Fintype X] [Fintype Y] [DecidableEq X] [DecidableEq Y]
  (k : Kernel X Y) (hk : Normalised k) :
  comp k (copy Y) = comp (copy X) (tensor k k) ↔ IsDeterministic k := by
  constructor
  · intro h x
    refine exists_pointMass_of_mul (k x) (hk x) fun y1 y2 => ?_
    have := congrFun (congrFun h x) (y1, y2)
    rw [comp_copy_apply, copy_comp_tensor_apply] at this
    exact this.symm
  · intro h
    funext x y
    rcases y with ⟨y1, y2⟩
    obtain ⟨a, ha⟩ := h x
    rw [comp_copy_apply, copy_comp_tensor_apply, ha y1, ha y2]
    exact (mul_lte_eq_of_pointMass a y1 y2).symm

/-- Deterministic kernels are exactly the 'ofFun f' (Julia 'deterministic(X, Y, f)'). -/
theorem isDeterministic_iff_exists_ofFun [DecidableEq Y] (k : Kernel X Y) :
  IsDeterministic k ↔ ∃ f : X → Y, k = ofFun f := by
  constructor
  · intro h
    choose f hf using h
    exact ⟨f, funext fun x => funext fun y => hf x y⟩
  · rintro ⟨f, rfl⟩ x
    exact ⟨f x, fun _ => rfl⟩

/-- Deterministic kernels are normalised. -/
theorem IsDeterministic.normalised [Fintype Y] [DecidableEq Y] {k : Kernel X Y}
  (h : IsDeterministic k) : Normalised k := by
  obtain ⟨f, rfl⟩ := (isDeterministic_iff_exists_ofFun k).mp h
  exact Normalised.ofFun f

```

Copy versus tensor for states

```

theorem isPointMass_iff_isDeterministic [DecidableEq X] (p : State X) :
  IsPointMass p ↔ IsDeterministic p := by
  constructor
  · rintro ⟨a, ha⟩ _; exact ⟨a, ha⟩
  · intro h; exact h ()

theorem isPointMass_iff_exists_pointMass [DecidableEq X] (p : State X) :
  IsPointMass p ↔ ∃ a, p = pointMass a := by
  constructor
  · rintro ⟨a, ha⟩
    exact ⟨a, funext fun u => funext fun x => by cases u; exact ha x⟩
  · rintro ⟨a, rfl⟩
    exact ⟨a, fun x => rfl⟩

/-- **Copy once is not two samples.** For a normalised state 'p', drawing once and copying
('compose(p, mcopy(X))') equals drawing twice independently ('otimes(p, p)'), with the domain
'I ⊗ I' identified with 'I' by 'copy Unit' iff 'p' is a point mass. -/
theorem comp_copy_eq_tensor_iff_isPointMass [Fintype X] [DecidableEq X] (p : State X)
  (hp : Normalised p) :
  comp p (copy X) = comp (copy Unit) (tensor p p) ↔ IsPointMass p := by
  rw [comp_copy_eq_iff_isDeterministic p hp, isPointMass_iff_isDeterministic]

```

```

/-- The same statement entrywise, without the ' $I \otimes I \cong I$ ' reindexing:
'copied.table == independent.table' iff 'p' is a point mass. -/
theorem comp_copy_eq_tensor_iff_isPointMass' [Fintype X] [DecidableEq X] (p : State X)
  (hp : Normalised p) :
  (∀ x x', comp p (copy X) () (x, x') = tensor p p ((), ()) (x, x')) ↔ IsPointMass p := by
  rw [← comp_copy_eq_tensor_iff_isPointMass p hp]
  constructor
  · intro h
    funext u y
    rcases y with ⟨x, x'⟩
    cases u
    rw [copy_comp_tensor_apply]
    exact h x x'
  · intro h x x'
    have := congrFun (congrFun h ()) (x, x')
    rw [copy_comp_tensor_apply] at this
    exact this

/-- Point masses do satisfy the equation (Julia: ' $\text{compose}(q, \text{mcopy}(X)) \approx \text{otimes}(q, q)$ ' for
' $q = \text{point\_mass}(X, 1)$ '). -/
theorem pointMass_comp_copy [Fintype X] [DecidableEq X] (a : X) :
  comp (pointMass a) (copy X) = comp (copy Unit) (tensor (pointMass a) (pointMass a)) :=
  (comp_copy_eq_tensor_iff_isPointMass _ (Normalised.pointMass a)).mpr ⟨a, fun _ => rfl⟩

/-- The uniform state on two points is not a point mass, so for it the equation fails
(the Julia ' $\text{state}(X, [0.2, 0.3, 0.5])$ ' example, in its smallest form). -/
example : ¬ IsPointMass (fun _ => (1 / 2 : ℝ) : State (Fin 2)) := by
  rintro ⟨a, ha⟩
  have := ha a
  norm_num at this

example :
  comp (fun _ => (1 / 2 : ℝ) : State (Fin 2)) (copy (Fin 2)) ≠
  comp (copy Unit) (tensor (fun _ => (1 / 2 : ℝ)) (fun _ => (1 / 2 : ℝ))) := by
  rw [Ne, comp_copy_eq_tensor_iff_isPointMass _ (fun _ => by simp)]
  rintro ⟨a, ha⟩
  have := ha a
  norm_num at this

end FiniteKernelsProofs.Finite.Kernel

```

FiniteKernelsProofs.Theory.Correspondence

```

import Mathlib.CategoryTheory.CopyDiscardCategory.Basic
import Mathlib.CategoryTheory.CopyDiscardCategory.Deterministic
import Mathlib.CategoryTheory.MarkovCategory.Basic

```

Dictionary between the GATlab theories used by MarkovCategories.jl (src/theory.jl) and Mathlib's CopyDiscardCategory / MarkovCategory, with the theorem of the finite model (Finite/Kernel.lean, Finite/Laws.lean) that instantiates each axiom.

This module spans two Julia packages: the finite model it instantiates the axioms in belongs to FiniteKernels.jl, which holds these proofs, while the GATlab side of the dictionary now lives in MarkovCategories.jl (its src/theory.jl and src/finstoch_model.jl); FiniteKernels.jl itself carries no Catlab or GATlab dependency and names the same operations compose_kernel, tensor_kernel, identity_kernel, copy_kernel, discard_kernel and swap_kernel.

ThCopyDiscardCategory is MarkovCategories.jl's alias for Catlab's ThMonoidalCategoryWithDiagonals (Catlab/src/theories/Monoidal.jl, "Cartesian category" section); ThMarkovCategory adds one axiom, the naturality of delete.

Comonoid structure

- $\text{Copy } \text{mcopy}(A) :: A \rightarrow A \otimes A$ (Δ) is `Mathlib's ComonObj.comul ( $\Delta[X]$ )` and the finite kernel `copy X`; delete is `ComonObj.counit ( $\epsilon[X]$ )` and `discard X`.
- Coassociativity $\Delta(A) \cdot (\Delta(A) \otimes \text{id}(A)) == \Delta(A) \cdot (\text{id}(A) \otimes \Delta(A))$ is `ComonObj.comul_assoc`, proved by `Kernel.copy_assoc`.
- The two counit laws are `ComonObj.counit_comul` and `ComonObj.comul_counit`, proved by `copy_discard_left` and `copy_discard_right` (with `_strict` variants).
- Cocommutativity $\Delta(A) \cdot \sigma(A, A) == \Delta(A)$ is `IsCommComonObj.comul_comm`, proved by `copy_swap`.

Tensor coherence

- $\Delta(A \otimes B) == (\Delta(A) \otimes \Delta(B)) \cdot (\text{id}(A) \otimes \sigma(A, B) \otimes \text{id}(B))$ is `CopyDiscardCategory.copy_tensor`, proved by `copy_prod`.
- $\diamond(A \otimes B) == \diamond(A) \otimes \diamond(B)$ is `CopyDiscardCategory.discard_tensor`, proved by `discard_prod`.
- $\Delta(\text{munit}()) == \text{id}(\text{munit}())$ and $\diamond(\text{munit}()) == \text{id}(\text{munit}())$ are `CopyDiscardCategory.copy_unit` and `CopyDiscardCategory.discard_unit`, proved by `copy_unit` and `discard_unit`.
- The inherited symmetric monoidal structure corresponds to `SymmetricCategory` and the laws `comp_tensor`, `tensor_assoc`, `tensor_unit_*`, `swap_swap`, `tensor_swap`, `hexagon`.

The Markov axiom and the non-axiom

- $f \cdot \diamond(B) == \diamond(A) \dashv [f :: (A \rightarrow B)]$ is `MarkovCategory.discard_natural`. In the finite model `comp_discard_eq_discard_iff` says it holds exactly when `Normalised`.
- Copy naturality $f \cdot \Delta(B) == \Delta(A) \cdot (f \otimes f)$ is **not an axiom**: it is the property `Deterministic f` (`IsComonHom f`), characterised by `comp_copy_eq_iff_isDeterministic`.

Catlab's tensor is strict, so the Mathlib associator, unitors and `tensorμ` are identities on the Julia side; in the finite model they are the reindexing kernels `assoc`, `leftUnitor`, ..., `tensorμ`. The generic consequences of these axioms (`discard_natural` as a theorem, `deterministic_comp`, `deterministic_copy`, `state_discard`) are stated once for the abstract Mathlib classes in the sibling `BayesianNetworks.jl` proof project's `Markov/Basic.lean` and are not repeated here. The examples below only check that each Mathlib field has the type the dictionary claims.

```
namespace FiniteKernelsProofs.Theory

open CategoryTheory MonoidalCategory ComonObj CopyDiscardCategory

universe v u

variable {C : Type u} [Category.{v} C] [MonoidalCategory.{v} C]

section CopyDiscard

variable [CopyDiscardCategory C]

-- Commutative comonoid axioms of 'ThMonoidalCategoryWithDiagonals'.
example (X : C) : Δ[X] >> (X < Δ[X]) = Δ[X] >> (Δ[X] > X) >> (α_ X X X).hom := comul_assoc X
example (X : C) : Δ[X] >> (ε[X] > X) = (λ_ X).inv := counit_comul X
example (X : C) : Δ[X] >> (X < ε[X]) = (ρ_ X).inv := comul_counit X
example (X : C) : Δ[X] >> (β_ X X).hom = Δ[X] := IsCommComonObj.comul_comm X

-- Coherence axioms.
-- ('Δ[X ⊗ Y]' must be read with the 'CopyDiscardCategory.comonObj' instance, not the generic
-- tensor product of comonoids 'Comon.instComonObjTensorObj'; the axiom says the two agree.)
example (X Y : C) :
  let I : ComonObj (X ⊗ Y) := CopyDiscardCategory.comonObj (X ⊗ Y)
  Δ[X ⊗ Y] = (Δ[X] ⊗_m Δ[Y]) >> tensorμ X X Y Y := copy_tensor X Y
example (X Y : C) :
  let I : ComonObj (X ⊗ Y) := CopyDiscardCategory.comonObj (X ⊗ Y)
  ε[X ⊗ Y] = (ε[X] ⊗_m ε[Y]) >> (λ_ (1_ C)).hom := discard_tensor X Y
example : Δ[1_ C] = (λ_ (1_ C)).inv := copy_unit
example : ε[1_ C] = 1_ (1_ C) := discard_unit

-- Copy naturality is not an axiom; it is the definition of 'Deterministic'.
```

```

example {X Y : C} (f : X → Y) [Deterministic f] : f » Δ[Y] = Δ[X] » (f ⊗m f) :=
  Deterministic.copy_natural f

end CopyDiscard

section Markov

variable [MarkovCategory C]

-- The one axiom 'ThMarkovCategory' adds.
example {X Y : C} (f : X → Y) : f » ε[Y] = ε[X] := MarkovCategory.discard_natural f

end Markov

end FiniteKernelsProofs.Theory

```

FiniteKernelsProofs.Theory.FinStoch

```

import FiniteKernelsProofs.Finite.Laws
import Mathlib.CategoryTheory.MarkovCategory.Basic

```

FinStoch as a Mathlib Markov category (SPEC §6, §10.3–§10.5).

Objects are finite types, morphisms are nonnegative normalised real kernels, and the tensor is the Cartesian product of state types with the independent product of kernels. Empty objects are permitted: there is no stochastic map from an inhabited type to an empty type, but this is not an obstruction to the category or its Markov structure.

The associators, unitors and symmetry are deterministic reindexings. Their coherence proofs reduce to equality of the underlying functions, not to additional axioms. In particular copy is **not** assumed natural for arbitrary kernels.

This packages the equations in `Finite/Kernel.lean`; it does not prove a correspondence with Julia floating-point arrays, and it does not make open Bayesian-network syntax a category.

```

set_option autoImplicit false

namespace FiniteKernelsProofs

open CategoryTheory Finite

/-- Finite state types with decidable equality, including the empty type. -/
structure FinStoch where
  carrier : Type
  [fintype : Fintype carrier]
  [decEq : DecidableEq carrier]

attribute [instance] FinStoch.fintype FinStoch.decEq

namespace FinStoch

instance instCategory : Category FinStoch where
  Hom X Y := {k : Kernel X.carrier Y.carrier // Kernel.Stochastic k}
  id X := ⟨Kernel.idK X.carrier, Kernel.Stochastic.ofFun id⟩
  comp k l := ⟨Kernel.comp k.1 l.1, k.2.comp l.2⟩
  id_comp k := Subtype.ext (Kernel.idK_comp k.1)
  comp_id k := Subtype.ext (Kernel.comp_idK k.1)
  assoc k l m := Subtype.ext (Kernel.comp_assoc k.1 l.1 m.1)

variable {X Y Z : FinStoch}

/-- A function induces a stochastic, deterministic morphism. -/
def det (f : X.carrier → Y.carrier) : X → Y :=
  ⟨Kernel.ofFun f, Kernel.Stochastic.ofFun f⟩

/-- A state-space equivalence induces an isomorphism of stochastic kernels. -/

```

```

def isoOfEquiv (e : X.carrier  $\approx$  Y.carrier) : X  $\cong$  Y where
  hom := det e
  inv := det e.symm
  hom_inv_id := Subtype.ext (by
    change Kernel.comp (Kernel.ofFun e) (Kernel.ofFun e.symm) = Kernel.idK X.carrier
    rw [Kernel.ofFun_comp_ofFun]
    exact Kernel.ofFun_congr fun x  $\Rightarrow$  e.symm_apply_apply x)
  inv_hom_id := Subtype.ext (by
    change Kernel.comp (Kernel.ofFun e.symm) (Kernel.ofFun e) = Kernel.idK Y.carrier
    rw [Kernel.ofFun_comp_ofFun]
    exact Kernel.ofFun_congr fun y  $\Rightarrow$  e.apply_symm_apply y)

instance instMonoidalCategoryStruct : MonoidalCategoryStruct FinStoch where
  tensorObj X Y := (X.carrier  $\times$  Y.carrier)
  tensorUnit := (Unit)
  tensorHom f g := (Kernel.tensor f.1 g.1, f.2.tensor g.2)
  whiskerLeft X _ _ g :=
    (Kernel.tensor (Kernel.idK X.carrier) g.1, (Kernel.Stochastic.ofFun id).tensor g.2)
  whiskerRight f Y :=
    (Kernel.tensor f.1 (Kernel.idK Y.carrier), f.2.tensor (Kernel.Stochastic.ofFun id))
  associator X Y Z := isoOfEquiv (Equiv.prodAssoc X.carrier Y.carrier Z.carrier)
  leftUnitor X := isoOfEquiv (Equiv.punitProd X.carrier)
  rightUnitor X := isoOfEquiv (Equiv.prodPUnit X.carrier)

open MonoidalCategory

instance instMonoidalCategory : MonoidalCategory FinStoch :=
  MonoidalCategory.ofTensorHom
  (id_tensorHom_id := fun _ _  $\Rightarrow$  Subtype.ext Kernel.tensor_idK)
  (id_tensorHom := by intros; rfl)
  (tensorHom_id := by intros; rfl)
  (tensorHom_comp_tensorHom := fun f g f' g'  $\Rightarrow$ 
    Subtype.ext (Kernel.comp_tensor f.1 g.1 f'.1 g'.1))
  (associator_naturality := fun f g h  $\Rightarrow$  Subtype.ext (Kernel.tensor_assoc f.1 g.1 h.1))
  (leftUnitor_naturality := fun f  $\Rightarrow$  Subtype.ext (Kernel.tensor_unit_left f.1))
  (rightUnitor_naturality := fun f  $\Rightarrow$  Subtype.ext (Kernel.tensor_unit_right f.1))
  (pentagon := by
    intro W X Y Z
    apply Subtype.ext
    change Kernel.comp (Kernel.tensor (Kernel.ofFun _) (Kernel.idK _))
      (Kernel.comp (Kernel.ofFun _) (Kernel.tensor (Kernel.idK _) (Kernel.ofFun _))) =
      Kernel.comp (Kernel.ofFun _) (Kernel.ofFun _)
    simp only [Kernel.idK, Kernel.tensor_ofFun, Kernel.ofFun_comp_ofFun]
    rfl)
  (triangle := by
    intro X Y
    apply Subtype.ext
    change Kernel.comp (Kernel.ofFun _)
      (Kernel.tensor (Kernel.idK _) (Kernel.ofFun _)) =
      Kernel.tensor (Kernel.ofFun _) (Kernel.idK _)
    simp only [Kernel.idK, Kernel.tensor_ofFun, Kernel.ofFun_comp_ofFun]
    rfl)

instance instSymmetricCategory : SymmetricCategory FinStoch where
  braiding X Y := isoOfEquiv (Equiv.prodComm X.carrier Y.carrier)
  braiding_naturality_left := by
    intro X Y f Z
    apply Subtype.ext
    exact Kernel.tensor_swap f.1 (Kernel.idK _)
  braiding_naturality_right := by
    intro X Y Z f
    apply Subtype.ext
    exact Kernel.tensor_swap (Kernel.idK _) f.1
  hexagon_forward := by
    intro X Y Z
    apply Subtype.ext

```

```

    exact Kernel.hexagon
hexagon_reverse := by
  intro X Y Z
  apply Subtype.ext
  change Kernel.comp (Kernel.ofFun _)
    (Kernel.comp (Kernel.ofFun _) (Kernel.ofFun _)) =
      Kernel.comp (Kernel.tensor (Kernel.idK _) (Kernel.ofFun _))
        (Kernel.comp (Kernel.ofFun _) (Kernel.tensor (Kernel.ofFun _) (Kernel.idK _)))
  simp only [Kernel.idK, Kernel.tensor_ofFun, Kernel.ofFun_comp_ofFun]
  rfl
symmetry := by
  intro X Y
  apply Subtype.ext
  exact Kernel.swap_swap

instance instComonObj (X : FinStoch) : ComonObj X where
  counit := ⟨Kernel.discard X.carrier, Kernel.Nonneg.discard, Kernel.Normalised.discard⟩
  comul := det fun x ⇒ (x, x)
  counit_comul := Subtype.ext Kernel.copy_discard_left
  comul_counit := Subtype.ext Kernel.copy_discard_right
  comul_assoc := Subtype.ext (by
    change Kernel.comp (Kernel.copy X.carrier)
      (Kernel.tensor (Kernel.idK _) (Kernel.copy _)) =
        Kernel.comp (Kernel.copy X.carrier)
          (Kernel.comp (Kernel.tensor (Kernel.copy _) (Kernel.idK _)) (Kernel.assoc _ _ _)))
    simp only [Kernel.comp_assoc] using (Kernel.copy_assoc (X := X.carrier)))

instance instIsCommComonObj (X : FinStoch) : IsCommComonObj X where
  comul_comm := Subtype.ext Kernel.copy_swap

open scoped ComonObj

/-- The structural 'tensorμ' of Mathlib is the middle-swap reindexing kernel. -/
theorem tensorμ_val (X Y : FinStoch) :
  (MonoidalCategory.tensorμ X X Y Y).val = Kernel.tensorμ X.carrier Y.carrier := by
  simp only [MonoidalCategory.tensorμ]
  change Kernel.comp (Kernel.ofFun _)
    (Kernel.comp (Kernel.tensor (Kernel.idK _) (Kernel.ofFun _))
      (Kernel.comp (Kernel.tensor (Kernel.idK _) (Kernel.tensor (Kernel.ofFun _) (Kernel.idK _)))
        (Kernel.comp (Kernel.tensor (Kernel.idK _) (Kernel.ofFun _)) (Kernel.ofFun _)))) = _
  simp only [Kernel.idK, Kernel.tensor_ofFun, Kernel.ofFun_comp_ofFun, Kernel.tensorμ]
  rfl

instance instMarkovCategory : MarkovCategory FinStoch where
  copy_tensor X Y := Subtype.ext (by
    change Kernel.copy (X.carrier × Y.carrier) =
      Kernel.comp (Kernel.tensor (Kernel.copy X.carrier) (Kernel.copy Y.carrier))
        (MonoidalCategory.tensorμ X X Y Y).val
    rw [tensorμ_val]
    exact Kernel.copy_prod)
  discard_tensor X Y := Subtype.ext Kernel.discard_prod
  copy_unit := Subtype.ext Kernel.copy_unit
  discard_unit := Subtype.ext Kernel.discard_unit
  discard_natural f := Subtype.ext (Kernel.Normalised.comp_discard f.2.2)

/-- The abstract Markov discard law uses exactly the normalisation carried by each hom. -/
theorem discard_natural (f : X → Y) : f > ε[Y] = ε[X] :=
  MarkovCategory.discard_natural f

/-- Copy naturality still characterises deterministic kernels; it is not a Markov axiom. -/
theorem copy_natural_iff (f : X → Y) :
  f > Δ[Y] = Δ[X] > (f ⊗m f) ↔ Kernel.IsDeterministic f.val := by
  rw [Subtype.ext_iff]
  exact Kernel.comp_copy_eq_iff_isDeterministic f.1 f.2.2

/-- Empty objects are allowed, but normalisation rules out a map into one from a nonempty

```

```

object. No unmentioned inhabitation assumption is used by the category instances. -/
theorem no_hom_to_empty [Nonempty X.carrier] [IsEmpty Y.carrier] (f : X → Y) : False := by
  obtain ⟨x⟩ := ⟨Nonempty X.carrier⟩
  have h := f.2.2 x
  simp at h

end FinStoch

end FiniteKernelsProofs

```

Roadmap

```
import FiniteKernelsProofs.Theory.FinStoch
```

The two former holes, `MonoidalCategory FinStoch` and `MarkovCategory FinStoch`, are now proved in `Theory/FinStoch.lean`, imported by the default target and included in the axiom audit. This compatibility module has no unproved declarations.

Remaining work is a representation bridge to Julia's named axes and floating-point arrays. The `Mathlib` instance by itself does not establish that bridge, and no open-network category or semantic functor is constructed here.

```

namespace FiniteKernelsProofs.Roadmap

/-- Compatibility name for the finite stochastic category now in the default library. -/
abbrev FinStoch := FiniteKernelsProofs.FinStoch

end FiniteKernelsProofs.Roadmap

```